

CrossSafe: Towards Cross-Embodiment Latent Safety Filters

Ihab Tabbara*, Yuxuan Yang*, and Hussein Sibai

Abstract—Cross-embodiment learning has shown that a single model, such as a vision-language-action (VLA) model, can learn state representations and manipulation skills that can be applied across heterogeneous robots to accomplish various tasks. We hypothesize that the same holds for safety enforcement. The reasoning required to satisfy a safety constraint, such as detecting an obstacle, recognizing that it should be avoided, and selecting a safe abstract action, is largely shared across robots. What differs across embodiments is how the abstract safe action is realized: morphology, kinematics, and dynamics determine which actions are safe and feasible. Consequently, the same action can be safe for one robot and unsafe for another. This is especially important for generalist manipulation policies that operate in a common end-effector action space without explicitly capturing how safety depends on the robot’s morphology and kinematics. We propose embodiment-conditioned safety filtering, in which a Hamilton–Jacobi reachability-based value function and its corresponding safety-maximizing policy are shared across robots. Using a morphology-aware latent representation of the robot and its environment, we perform Hamilton–Jacobi reachability analysis directly in latent space so that the learned safety concepts can generalize across embodiments while remaining explicitly conditioned on each robot’s morphology and kinematics. We evaluate our approach across five bimanual robot embodiments and five manipulation tasks with whole-body collision-avoidance constraints. Our results show that a single policy, jointly trained across five manipulation tasks and four embodiments, exhibits zero-shot generalization to a held-out embodiment, reducing the nominal policy’s collision rate. They also show that training using more embodiments improves generalization.

I. INTRODUCTION

Scaling robot learning by collecting and training on separate datasets for every robot platform is costly and limits the reuse of experience across embodiments. Cross-embodiment learning addresses this problem by sharing task knowledge across robots with different morphologies and control interfaces. This idea has been explored across multi-robot policies [1], [2], [3], morphology-aware manipulation models [4], [5], [6], and cross-gripper or dexterous-hand generalization [7], [8]. These results suggest that substantial task knowledge can be shared across physically different robots. We ask whether the same principle can be extended to safety filters: can data collected from multiple robot embodiments be used to learn a safety filter that generalizes to a novel one?

Many generalist and cross-embodiment manipulation policies output actions in a common task-space representation across robots, such as delta end-effector pose commands [2], [9], which can then be mapped to embodiment-specific

joint motions. While this action representation lets a single policy’s manipulation skills transfer across embodiments, the same end-effector trajectory can induce different whole-body motions on robots with different link lengths, joint axes, joint limits, and configurations. Consequently, the same action may be safe for one embodiment but unsafe for another, and the safe control required to avoid entering the failure set may be embodiment-dependent.

This motivates a cross-embodiment state representation that preserves the safety-relevant information shared across robots while retaining sufficient information about the morphology and kinematics of each robot, together with an action space that enables different safe whole-body controls for different robots.

Existing cross-embodiment safeguarding methods address this differently: EmbodiSteer [10] corrects a given diffusion policy’s actions at each denoising step using embodiment-specific kinematics and whole-body collision costs computed with cuRobo, while Any-Body Guard [11] computes safe actions in each robot’s native configuration space. Neither learns a safety representation that supports jointly training a single safety value function and its corresponding safety-maximizing policy across heterogeneous robot embodiments. We therefore study embodiment-conditioned safety filtering, where a single Hamilton–Jacobi (HJ) value function is learned for heterogeneous robot embodiments. We seek a representation that captures safety-relevant information shared across robots while preserving morphology- and state-dependent information relevant for safety.

We build on HoloBrain-0 [5], whose pretrained representation provides a natural starting point as it jointly encodes multi-view 3D scene information and variable robot kinematic structures using joint poses and graph-structured attention. We augment HoloBrain-0’s representation with safety-relevant per-link kinematic features and introduce geometry-aware manipulator-scene cross-attention that incorporates distances and directions between robot links and scene patches, expressed in a shared 3D coordinate frame. The resulting embodiment-conditioned latent state allows training a shared safety critic to generalize across embodiments and assign different safety values to different embodiments. A corresponding policy can also be trained to produce embodiment-dependent safety-maximizing actions.

We evaluate our approach in RoboTwin 2.0 [12] across five bimanual embodiments and five manipulation tasks under collision-avoidance constraints. Using five leave-one-embodiment-out folds, we jointly train the HJ value function and safe policy on four embodiments and evaluate them on the fifth, which is excluded from training of the

*Equal contribution.

The authors are with the Department of Computer Science and Engineering, Washington University in St. Louis, St. Louis, MO, USA. {i.k.tabbara, y.yuxuan, sibai}@wustl.edu

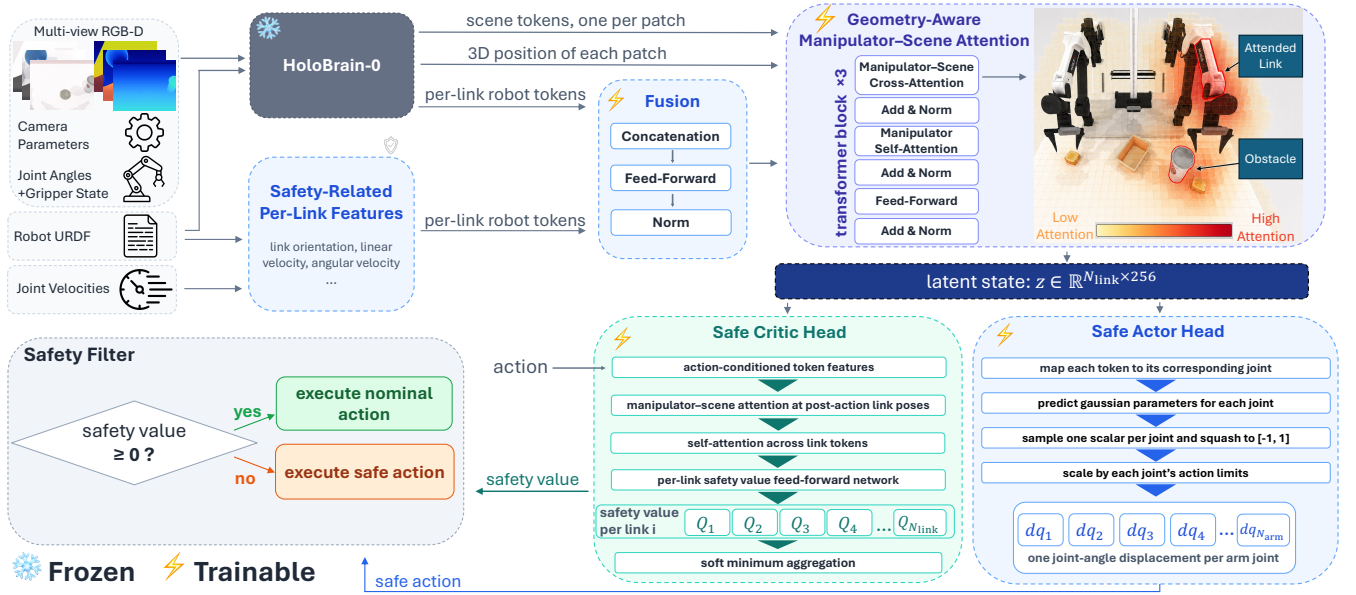


Fig. 1: **Overview of CrossSafe.** A frozen HoloBrain-0 produces scene and per-link robot tokens, which we augment with safety-related per-link features not encoded by HoloBrain-0. Geometry-aware cross-attention produces a safety-aware latent state with one token per robot link. The safety critic and actor use this latent state to estimate safety and output per-joint angle displacements, respectively, generalizing across embodiments with different degrees of freedom (DoFs).

safety-filter components. To our knowledge, CrossSafe is the first framework to jointly learn a single latent safety filter across multiple robot embodiments and demonstrate zero-shot generalization to embodiments excluded from safety-filter training. Our contributions are:

- We introduce a Hamilton-Jacobi reachability-based safety-maximizing actor and critic whose latent state encodes a robot’s proprioception, observations, and morphology, enabling generalization across embodiments, even ones with different degrees of freedom.
- We evaluate CrossSafe on five bimanual robot embodiments and five tasks in RoboTwin 2.0, showing reduced collision rates on unseen embodiments without fine-tuning, and improved generalization when training on data from four embodiments instead of a single one.

II. RELATED WORK

A. Cross-embodiment generalist policies

Large-scale multi-robot datasets and generalist policies such as Open X-Embodiment [1], Octo [2], OpenVLA [9], $\pi_{0.5}$ [13], and CrossFormer [3] show the benefits of training across heterogeneous robot platforms. Recent cross-embodiment policies address embodiment variation through domain-specific prompts [4], morphology conditioning [6], shared action representations [14], geometric interfaces [15], end-effector traces [16], or shared embodied reasoning [17]. Cross-embodiment generalization has also been studied for robot hands with heterogeneous end effectors [7], [8].

A complementary line of work explicitly encodes the robot’s morphology. HoloBrain-0 [5], which we build on, combines multi-view 3D perception with URDF-derived

kinematic priors, joint poses, and graph-structured attention over the kinematic chain. We extend this representation to also encode safety-critical features. Related methods encode morphology through kinematic graphs [18], a topology-aware end-effector graph paired with geometry-aware state tokens [19], morphology-agnostic encoder-decoder architectures for multi-embodiment locomotion [20], or morphology-conditioned world models [21].

Our goal is to generalize safety assurance across embodiments. In current VLA design practice, cross-embodiment pretraining is commonly followed by adaptation or fine-tuning to the target robot [4], [5], [9], [13], [2]. Recent methods such as LAP [22] and Cloak [23] instead target zero-shot transfer by promoting embodiment-invariant task representations, but such invariance is less suitable for safety-critical control, where robot morphology directly affects the set of possible safe actions. We therefore seek a state representation that generalizes across robots while preserving the embodiment-specific information needed for safe control.

B. Latent safety filters and safe control

Control barrier functions (CBFs) [24] and Hamilton-Jacobi (HJ) reachability [25] provide principled tools for synthesizing safety filters, while learning-based methods extend both frameworks to higher-dimensional systems, through neural CBFs [26], [27] and through HJ methods that approximate the value function with reinforcement learning [28]. Recent works extend safety filtering to learned latent representations [29], [30], neural operators [31], learned configuration-space barriers [32], reachability-based learned policies [33], and temporal-logic specifications [34]. Language-conditioned HJ safety filters share one

learned actor and critic across multiple language-specified safety constraints [35]. A complementary line of work uses conformal prediction to bound the errors of learned safety filters, recovering probabilistic safety guarantees [36], [37], [38], [39], [40]. These advances generalize safety filtering across observations, constraints, environments, or specifications but do not address training a shared safety value function that is explicitly conditioned on robot embodiments.

More closely related to our work, EmbodiSteer [10] corrects the diffusion policy’s generated action at each denoising step using embodiment-specific robot kinematics and whole-body collision costs computed with cuRobo. Any-Body Guard [11] certifies a local probabilistically safe polytope in each robot’s configuration space, the space of joint angles $q \in \mathbb{R}^{N_{\text{Dof}}}$ with one coordinate per joint, by sampling configurations and evaluating a violation function built from that robot’s forward kinematics and an object-based scene representation. This is a sampling-based technique that does not train any models. It yields a probabilistic safety guarantee, but each robot’s safe set is recomputed from its own kinematic model at runtime, so no safety reasoning is shared across embodiments. We instead amortize this into a safety value function learned offline jointly across robots.

III. PRELIMINARIES

Hamilton–Jacobi (HJ) reachability is a formal control-theoretic framework for verifying control systems’ safety and synthesizing safe controllers [25]. Consider a dynamical system of the form $s_{t+1} = f(s_t, a_t)$, where $s_t \in S$ and $a_t \in A$ are the state and the action at time t , respectively. We denote the trajectory of the system starting from state s and following a policy $\pi : S \rightarrow A$ by $\xi_s^\pi : \mathbb{N}_{\geq 0} \rightarrow S$. Given a set of states $\mathcal{F} := \{s \mid h(s) < 0\}$ consisting of the *failure* (or *avoid*) states, where $h : S \rightarrow \mathbb{R}$ is a Lipschitz continuous function, HJ reachability analysis computes the optimal value function $V : S \rightarrow \mathbb{R}$ for *avoiding* $\mathcal{F}$, where $V(s) := \sup_\pi \inf_{t \geq 0} h(\xi_s^\pi(t))$. The latter is the fixed point of the Bellman equation: $V(s) = \min \{h(s), \max_{a \in A} V(f(s, a))\}$.

The associated optimal policy π^* for *avoiding* $\mathcal{F}$ satisfies $\forall s \in S, \pi^*(s) := \arg \max_{a \in A} V(f(s, a))$. The zero-sublevel set of V , i.e., the set $\{s \mid V(s) < 0\}$, is called the *backward reachable set* (BRS) of the system corresponding to $\mathcal{F}$. It consists of the states starting from which the system will inevitably reach the failure set eventually under any policy, and thus is the largest set of *unsafe* states.

Reinforcement learning-based approaches have been suggested to address the curse of dimensionality of HJ reachability [28]. Actor-critic algorithms (e.g., SAC [41]) were used for systems with continuous action spaces. In such approaches, the parameters θ of the Q-function, which is also called the *HJ safety critic*, are optimized by minimizing the loss function:

$$L(\theta) := \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} [(Q_\theta(s_t, a_t) - y_t)^2], \quad (1)$$

where $\mathcal{D}$ is the distribution of transitions (s_t, a_t, s_{t+1}) collected during environment rollouts and the *target*

y_t is defined as follows: $y_t := (1 - \gamma)h(s_t) + \gamma \min \{h(s_t), \max_{a \in A} Q_\theta(s_{t+1}, a)\}$ with $\gamma \in (0, 1)$ [28].

IV. MORPHOLOGY-AWARE LATENT SAFETY FILTERING

We learn a shared HJ safety critic Q_θ and a shared safe policy π_ϕ^{safe} across embodiments, potentially with different numbers of links and joints. Both operate on a morphology-aware latent state produced by an encoder, which builds on the frozen, pretrained HoloBrain-0 [5] vision and robot-state encoders. Our method, **CrossSafe**, is shown in Fig. 1.

To enable cross-embodiment safety filtering, we make four key design choices: (i) The actor outputs joint-angle displacements rather than end-effector pose deltas, directly specifying changes to the arms’ joint configurations. (ii) We augment HoloBrain-0’s per-link robot tokens with safety-related features that were not explicitly encoded by HoloBrain-0, including each link’s Cartesian linear and angular velocities and normalized depth in the kinematic tree. (iii) We let each robot-link token attend to scene patches while explicitly accounting for their spatial relationship in 3D. Particularly, in addition to the standard attention score based on the link and scene features, we bias the score using the distance and direction from the link to each scene patch in a shared world frame. (iv) We design the HJ actor and critic to operate on variable-length sequences of robot link tokens, allowing the same learned modules to be applied across manipulators with different numbers of links, joints, and degrees of freedom.

A. Frozen HoloBrain-0

Given an observation comprising multi-view RGB-D images, joint angles, gripper openings, and camera intrinsics and extrinsics, the frozen pretrained GroundingDINO variant of HoloBrain-0 produces two token streams: *scene tokens* encoding the environment and *per-link robot tokens* encoding the robot’s configuration. The scene encoder fuses features of the RGB images generated by a Swin-Transformer with features of the depth maps generated by another Swin-Transformer, and back-projects the result through the camera parameters into a shared 3D world frame, yielding scene tokens $\mathbf{c} = (c_1, \dots, c_{N_{\text{scene}}})$, $c_j \in \mathbb{R}^{256}$ and their corresponding 3D positions $\mathbf{p}^{\text{scene}} = (p_1^{\text{scene}}, \dots, p_{N_{\text{scene}}}^{\text{scene}})$, $p_j^{\text{scene}} \in \mathbb{R}^3$. Forward kinematics converts the joint angles into link positions and orientations. The robot state encoder embeds these poses together with the states of the grippers and applies self-attention informed by the robot’s kinematic graph, producing the tokens $\hat{\mathbf{z}} = (\hat{z}_1, \dots, \hat{z}_{N_{\text{link}}})$, $\hat{z}_i \in \mathbb{R}^{256}$. The joint angles are excluded, since they are inconsistent across embodiments with different zero-position definitions, rotation directions, and URDFs, while link poses provide a unified geometric reference [5]. Here $N_{\text{link}} = N_{\text{arm}} + N_{\text{gripper}}$ counts the moving links, where N_{arm} is the total number of moving arm links, which is equal to the number of actuated arm joints across both arms, and N_{gripper} is the number of grippers, each counted as a single link. The robot tokens encode the link poses and kinematic structure, while the scene tokens encode the observed environment. However,

such encoded information might not be sufficient and more features might be needed for safety enforcement.

B. Augmenting safety-related features

HoloBrain-0's robot state tokens are extracted from a single observation and do not explicitly encode the link velocities. Velocities are relevant to safety because avoiding contact depends on the robot's morphology, its link poses, and their motion relative to obstacles. We therefore augment each link token with a safety-related feature vector encoding its velocity, scale-normalized position, orientation, normalized depth in the kinematic tree, and arm identity.

We construct safety-related feature vectors separately for each arm from its URDF and its measured joint angles and velocities, then concatenate the features of the two arms into a single sequence for the bimanual robot. From each URDF, we extract the joint order and axes and estimate each arm's reach L_{reach} as the maximum end-effector-to-base distance. We form one token per link and merge paired gripper fingers into a single token carrying their mean position and the wrist orientation. For link i , the safety-related feature vector is

$$b_i = \left[\frac{p_i^{link} - p^{base}}{L_{reach}}, \eta_i^{wxyz}, \frac{J_i^{lin} \dot{q}}{v_0}, \frac{J_i^{ang} \dot{q}}{\omega_0}, \delta_i, \chi_i, g_i \right] \in \mathbb{R}^{16}, \quad (2)$$

where p_i^{link} is the world-frame position of the end of link i that is located at the joint connecting it to its parent, and p^{base} is the position of the arm base corresponding to the link, η_i^{wxyz} is the orientation of link i as a quaternion, $J_i^{lin}, J_i^{ang} \in \mathbb{R}^{3 \times N_{arm}}$ are the linear and angular velocity Jacobians of link i , and $\dot{q} \in \mathbb{R}^{N_{arm}}$ is the vector of measured joint velocities concatenated across both arms. $\delta_i \in (0, 1]$ is the normalized link depth in the kinematic tree and equals 1 for a leaf link, $\chi_i \in \{0, 1\}$ is the arm identity for a bimanual manipulator, and $g_i \in [0, 1]$ is the normalized gripper opening.

We use linear and angular link velocities expressed in a common world frame instead of the joint velocities $\dot{q}$, because the same $\dot{q}$ can produce different link motions across arms with different morphologies. We express each link position relative to its arm base and divide by L_{reach} , so that geometrically similar configurations on arms of different link lengths map to similar values. $v_0 = 0.5 \text{ m/s}$ and $\omega_0 = \pi \text{ rad/s}$ are normalization constants. The normalized depth δ_i makes link depth comparable across embodiments.

We fuse each token $\hat{z}_i$ from HoloBrain-0's frozen encoder with our safety-related feature vector b_i from Eq. (2) using a fusion module to obtain $\mathbf{z}^{(0)} = (z_1^{(0)}, \dots, z_{N_{link}}^{(0)})$, where

$$z_i^{(0)} = \text{Norm}\left(\text{MLP}\left(\text{Norm}(\hat{z}_i) \parallel b_i\right)\right) \in \mathbb{R}^{256}, \quad (3)$$

and MLP is a two-layer network. The same fusion weights are used for every link across all embodiments.

C. Geometry-aware manipulator-scene attention

The link and scene tokens initially encode the robot and the environment separately. To support safety reasoning, each link token needs to encode scene information. This is important for assessing the spatial relationship between the robot and nearby objects and obstacles. We connect the

two token streams through cross-attention, allowing each link token to encode scene features. Standard cross-attention weights scene patches without explicitly accounting for their spatial relation to the link. We instead add a learned attention bias based on the 3D distance and direction from each link to each scene patch, computed from their positions in a shared world frame. This enables the attention mechanism to jointly consider each scene patch's features and its spatial relationship to each robot link.

For the ℓ -th cross-attention layer with H heads, the queries of the m -th head are obtained from the link tokens and the keys and values are obtained from the scene tokens, i.e.,

$$q_i^{(m)} = W_q^{(m)} z_i^{(\ell-1)}, \quad k_j^{(m)} = W_k^{(m)} c_j, \quad v_j^{(m)} = W_v^{(m)} c_j, \quad (4)$$

so each link token is updated to encode scene content while the scene tokens are left unchanged. For link token i and scene token j , let $r_{ij} = p_j^{scene} - p_i^{link}$ be the displacement from the link to the patch, $d_{ij} = \|r_{ij}\|$ be the distance between them, and $\hat{r}_{ij} = r_{ij}/d_{ij}$ be the normalized displacement representing the direction from the link towards the patch. We bias the attention mechanism with these quantities and augment its output with the direction as follows:

$$\beta_{ij}^{(m)} = \text{softmax}_j \left[\frac{q_i^{(m)\top} k_j^{(m)}}{\sqrt{C_{head}}} + f_\psi^{(m)}(\log d_{ij}, \hat{r}_{ij}) \right], \quad (5)$$

$$o_i = W_o \left[\parallel_{m=1}^H \sum_j \beta_{ij}^{(m)} v_j^{(m)} \parallel \parallel_{m=1}^H \sum_j \beta_{ij}^{(m)} \hat{r}_{ij} \right], \quad (6)$$

where C_{head} is the feature dimension of each attention head, and $f_\psi^{(m)}$ produces the geometric attention bias for head m from the distance and direction between link i and scene patch j . The operator $\parallel_{m=1}^H$ denotes concatenation across heads, and W_o projects the concatenated features to the same dimension as that of $z_i^{(\ell-1)}$, allowing o_i to be added to $z_i^{(\ell-1)}$ through a residual connection. The second concatenated component in Eq. (6) is the attention-weighted direction from the link to the scene patches. It explicitly preserves directional information alongside the scene features.

Denoting Eqs. (4)–(6) by CA (for cross-attention), self-attention over the link tokens by SA, a feed-forward layer by FFN, and $\mathbf{p}^{link} = (p_1^{link}, \dots, p_{N_{link}}^{link})$, block $\ell = 1, 2, 3$ updates the tokens by:

$$\begin{aligned} \tilde{\mathbf{z}}^{(\ell)} &= \mathbf{z}^{(\ell-1)} + \text{CA}(\text{Norm}(\mathbf{z}^{(\ell-1)}), \mathbf{c}, \mathbf{p}^{link}, \mathbf{p}^{scene}), \\ \mathbf{z}^{(\ell)} &= \tilde{\mathbf{z}}^{(\ell)} + \text{SA}(\text{Norm}(\tilde{\mathbf{z}}^{(\ell)})), \\ \mathbf{z}^{(\ell)} &= \mathbf{z}^{(\ell)} + \text{FFN}(\text{Norm}(\mathbf{z}^{(\ell)})). \end{aligned} \quad (7)$$

The cross-attention aggregates scene-token information into each link token, weighting each scene patch jointly by its feature similarity to the link and by their relative positions. Self-attention propagates information across link tokens. We define the *latent state* to be:

$$\mathbf{z} = (z_1^{(3)}, \dots, z_{N_{link}}^{(3)}). \quad (8)$$

Accordingly, we define $\forall i, z_i := z_i^{(3)}$, for simplicity of notation.

D. Cross-embodiment safety critic and safe policy

a) *Safe policy* π_ϕ^{safe} : The safe policy takes the latent state $\mathbf{z}$ as input. A linear head, shared across all arm joints, maps each joint-associated link token to the mean and log standard deviation of a Gaussian distribution. We sample an action for each joint, squash it to $[-1, 1]$ using $\tanh$, and scale it by the corresponding action limit $a_i^{\text{max}} = \dot{q}_i^{\text{lim}} \Delta t$, where $\dot{q}_i^{\text{lim}}$ is the URDF-provided velocity limit and Δt is the control period. Gripper tokens do not produce actions as the gripper is controlled only by the nominal policy.

b) *Hamilton–Jacobi safety critic*: The critic evaluates a candidate action $a = [a_1, \dots, a_{N_{\text{arm}}}]^\top$, whose components are joint-angle displacements. Using the link Jacobians, we compute approximations of corresponding linear and angular displacements as $dp_i = J_i^{\text{lin}} a \in \mathbb{R}^3$ and $dw_i = J_i^{\text{ang}} a \in \mathbb{R}^3$.

For each arm link i , we also denote the command for its connecting joint by dq_i . To condition the critic on the candidate action, we augment each link token with dq_i , dp_i , and dw_i , which describe how the action would move that link. We concatenate these features with an indicator $\mathbf{1}_i$, which evaluates to one for links whose joints are controlled by the safe policy and to zero otherwise. A projection layer maps the resulting 8D vector into the token space:

$$\tilde{z}_i^{(0)} = z_i + W_a[dq_i, dp_i^\top, dw_i^\top, \mathbf{1}_i]^\top + b_a, \quad (9)$$

where $W_a \in \mathbb{R}^{256 \times 8}$ and $b_a \in \mathbb{R}^{256}$ are learned parameters shared across links and embodiments. For gripper tokens, we set $dq_i = \mathbf{1}_i = 0$, since the gripper is not controlled by a joint-angle displacement but by a separate opening command issued by the nominal policy; we retain dp_i and dw_i to describe how the arm moves the gripper.

The critic then applies two additional transformer blocks ($\ell = 1, 2$) of the form described in Eq. (7). Cross-attention combines the action-conditioned tokens with scene information, using the estimated post-action link positions $p_i^{\text{link}} + dp_i$ to compute the geometric bias in Eq. (5). This allows attention to account for how the action would change the distance and direction from each link to each scene patch. Self-attention shares this information across link tokens.

A two-layer feed-forward neural network g_θ , shared across tokens and embodiments, maps each updated token to a safety score $Q_i = g_\theta(\tilde{z}_i^{(2)})$, where $\tilde{z}_i^{(2)}$ is the updated representation of link token i after the critic’s two transformer blocks. We aggregate scores using a soft minimum:

$$Q_\theta(\mathbf{z}, \mathbf{c}, \mathbf{p}^{\text{link}}, \mathbf{p}^{\text{scene}}, a) = -T \log \sum_i \exp\left(-\frac{Q_i}{T}\right), \quad (10)$$

where $\mathbf{c}, \mathbf{p}^{\text{link}}, \mathbf{p}^{\text{scene}}$ are inputs to the critic’s transformer blocks. We write $Q_\theta(\mathbf{z}, a)$ from now on for brevity. Temperature $T > 0$ controls the smoothness of the minimum.

V. CROSS-EMBODIMENT EVALUATION SETUP

A. Experimental setup

We evaluate CrossSafe in RoboTwin 2.0 [12] across five bimanual embodiments: Piper, Franka-Panda, ARX-X5, UR5-WSG, and Aloha-AgileX. Franka-Panda has seven

DoFs per arm, while the others have six. We use the RoboTwin 2.0 checkpoint of the GroundingDINO variant of HoloBrain-0 [5] and keep its encoders frozen. We consider the five manipulation tasks shown in Fig. 2, preserving their objectives while adding the same static RoboTwin-OD Box Drink obstacle to each scene. During training, the obstacle is placed along the nominal path with probability 0.7 and off the path otherwise. During evaluation, it is always placed along the path. The nominal controller is RoboTwin 2.0’s cuRobo planner, which plans excluding the added obstacle.

We define $h(s)$ as the minimum signed distance between the obstacle and the robot and any object it is grasping. A set of enclosing spheres approximates the robot’s body, and a bounding box approximates the obstacle. Thus, $h(s) < 0$ indicates overlap between the geometric approximations, which can occur without physical contact.

B. Training pipeline

We jointly optimize all trainable components of CrossSafe shown in Fig. 1. Each model is trained on data from all five tasks and from its training embodiments, and evaluated on all five embodiments. A shared warmup buffer contains 20 nominal trajectories per embodiment-task pair, totaling 500 trajectories, including collision and collision-free episodes. Each model is trained on the trajectories from its training embodiments. All models use the same hyperparameters and collect online trajectories during training at the same rate.

We use soft actor-critic [41] adapted to the HJ reachability case with the loss described in Eq. (1), with a batch size of 64, a learning rate of 3×10^{-4} , and a discount factor of $\gamma = 0.9$. We use $T = 0.4$. Each model is trained for two days on one NVIDIA A40 GPU with 48 GB GPU memory, 8 CPU cores, and 48 GB RAM. More implementation details and all hyperparameters are in the Appendix.

C. Online safety filtering

The nominal controller and the safety filter run synchronously at each control step (25 Hz). The nominal controller proposes an action a_{nom} , which the safety critic evaluates as $Q_\theta(\mathbf{z}, a_{\text{nom}})$. The executed action is then

$$a = \begin{cases} a_{\text{nom}}, & Q_\theta(\mathbf{z}, a_{\text{nom}}) \geq 0, \\ \pi_\phi^{\text{safe}}(\mathbf{z}), & Q_\theta(\mathbf{z}, a_{\text{nom}}) < 0. \end{cases} \quad (11)$$

Whenever the safety filter intervenes, the nominal controller replans from the state reached after executing the safe action. On an NVIDIA A40, the HoloBrain-0 vision and robot-state encoders take 59.8 ± 1.4 ms per control step. Our added trainable modules (shown in Fig. 1) take 13.1 ± 0.3 ms. A full inference for the pipeline in Fig. 1 takes 73.0 ± 1.9 ms.

D. Baselines and ablations

We compare **CrossSafe** with the unfiltered nominal controller and three ablations: **CrossSafe w/o Aug** removes the safety-related feature augmentation described in Sec. IV-B. **CrossSafe w/o Geo** replaces geometry-aware cross-attention with plain cross-attention in the encoder and critic modules. **CrossSafe w/o Geo, Aug** applies both changes.

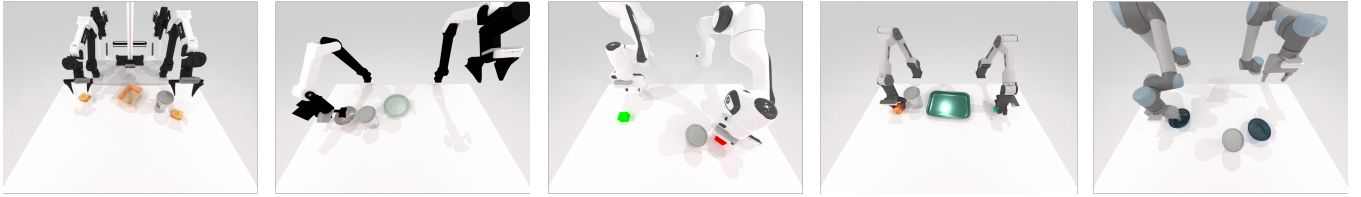


Fig. 2: Tasks from left to right: Place Bread in Basket, Place Container on Plate, Stack Two Blocks, Place Burger & Fries, and Stack Two Bowls. Embodiments from left to right: Aloha-AgileX, ARX-X5, Franka-Panda, Piper, and UR5-WSG.

We additionally instantiate our method by replacing HoloBrain-0’s frozen encoder with that of X-VLA [4], another state-of-the-art VLA. X-VLA encodes observations with a Florence-2 backbone that takes three camera views and a task instruction. Two properties of X-VLA’s representation prevent us from directly reusing the fusion module and geometry-aware attention from Sections IV-B and IV-C. First, X-VLA’s tokens do not explicitly include metric 3D position information. Second, X-VLA does not provide the link poses that our safety-related feature vector in Eq. (2) and fusion step in Eq. (3) are built on. It only exposes the end-effector proprioception. In place of the link tokens, the critic and policy operate on two per-arm tokens, obtained by encoding raw end-effector proprioception with a small MLP. These arm tokens attend to the scene tokens through cross-attention, structurally analogous to the manipulator-scene attention in Eqs. (4)-(7) but without being biased by the geometric information. The actor predicts a bounded change in end-effector position and orientation for each arm, while the critic produces a per-arm safety value pooled by Eq. (10), now taken over the two arm tokens rather than over link tokens. End-effector commands are converted into joint-angle displacements using damped least-squares inverse kinematics and executed when the safety filter intervenes. We call this variant **X-VLA-Safe**.

E. Metrics and evaluation protocol

We evaluate each trained model over 50 episodes per task-embodiment pair. All models are trained across the same five tasks. In-distribution (InD) evaluation uses embodiments whose data were used to train the model. Out-of-distribution (OOD) evaluation uses embodiments excluded from this training, though they may have appeared during the original training of the HoloBrain-0 and X-VLA encoders. We use collision rate (CR), success rate (SR), intervention rate (IR), and contact force (Force) as our metrics. CR is the percentage of the 50 evaluation episodes containing at least one physical contact, detected by the simulator, between the manipulator (or an object it holds) and the added obstacle, instead of bounding boxes’ intersections. SR is the percentage of these episodes in which the task is accomplished. To compute IR, we first calculate the percentage of control steps using the safe action within each episode, then average these percentages across the 50 episodes. Force is measured, in newtons, between the manipulator (or an object it holds) and the added obstacle. The simulator reports contact impulses. At each frame, we divide each contact impulse magnitude by

the timestep duration to obtain the corresponding force magnitude averaged over that timestep, and take the maximum over these contact points. For each episode, we compute the median of these frame-level values using only frames in which contact occurs, and assign zero to collision-free episodes. Force is then the average of these episode-level values over the 50 episodes.

For each method (X-VLA-Safe; CrossSafe w/o Geo, Aug; CrossSafe w/o Geo; CrossSafe w/o Aug; and CrossSafe), we use five leave-one-embodiment-out folds. Each fold trains a model on four embodiments and evaluates it on those four and the held-out fifth. Since each embodiment is held out in exactly one fold, every task-embodiment pair is evaluated in all five folds: four times as InD and once as OOD. In Table I, each model is evaluated on 20 InD task-embodiment pairs (1,000 episodes) and 5 OOD pairs (250 episodes). For each model, we average the CR, SR, IR, and Force values computed for each task-embodiment pair across all pairs evaluated by that model, separately for InD and OOD. For each method, we then report the mean and standard deviation of the averages across its five trained models, one model per leave-one-embodiment-out fold. Together, these five models are evaluated over 5,000 InD and 1,250 OOD episodes.

Table II compares the five CrossSafe models from Table I, termed *generalists*, with five CrossSafe *specialists*. Each generalist is trained on a different combination of four embodiments, whereas each specialist is trained on one embodiment. Each specialist is evaluated on its training embodiment (InD) and the other four embodiments (OOD), giving 5 InD task-embodiment pairs (250 episodes) and 20 OOD pairs (1,000 episodes). Each generalist is evaluated on its four training embodiments (InD) and the held-out fifth embodiment (OOD), giving 20 InD pairs (1,000 episodes) and 5 OOD pairs (250 episodes). For each model, we average the metrics across its evaluated task-embodiment pairs, separately for InD and OOD. We then report the mean and standard deviation of these averages separately across the five generalists and the five specialists. Specialists are evaluated over 1,250 InD and 5,000 OOD episodes, and generalists over 5,000 InD and 1,250 OOD episodes.

Detailed results for each method on every task-embodiment pair, separately for InD and OOD evaluations, are in the Appendix.

VI. RESULTS

a) *A single filter improves safety across multiple tasks and embodiments:* From Table I, CrossSafe reduces the

Method	In-distribution Embodiments				Out-of-distribution Embodiment			
	CR (%) ↓	SR (%) ↑	IR (%)	Force [N] ↓	CR (%) ↓	SR (%) ↑	IR (%)	Force [N] ↓
Nominal	64.1	32.2	—	179.6	64.1	32.2	—	179.6
X-VLA-Safe	50.4 ± 0.7	17.0 ± 4.1	5.2 ± 0.8	189.7 ± 30.3	53.8 ± 17.4	16.7 ± 12.1	4.8 ± 1.5	247.0 ± 144.6
CrossSafe w/o Geo, Aug	38.3 ± 7.7	31.3 ± 4.1	11.1 ± 7.0	81.3 ± 18.0	51.5 ± 18.6	35.0 ± 15.7	7.6 ± 4.3	126.2 ± 94.8
CrossSafe w/o Geo	45.4 ± 7.4	23.8 ± 5.9	20.2 ± 4.6	60.2 ± 17.4	53.4 ± 17.7	30.0 ± 11.5	15.5 ± 10.4	79.0 ± 53.7
CrossSafe w/o Aug	37.1 ± 5.0	26.0 ± 7.5	7.7 ± 1.8	150.1 ± 35.8	49.7 ± 13.3	23.3 ± 13.1	9.2 ± 8.8	133.5 ± 31.8
CrossSafe	39.8 ± 3.8	23.7 ± 2.8	13.2 ± 1.2	113.6 ± 41.9	49.8 ± 8.3	29.4 ± 15.7	9.2 ± 6.1	102.9 ± 44.3

TABLE I: Comparison of methods using CR, SR, IR, and Force. Each learned method has five trained models, one per leave-one-embodiment-out fold. Each model is evaluated on its four training embodiments (InD) and held-out fifth (OOD).

	Model	CR (%) ↓	SR (%) ↑	IR (%)	Force [N] ↓
InD Embodiment(s)	Specialist	37.6 ± 21.3	18.7 ± 10.2	11.2 ± 4.8	113.0 ± 87.1
	Generalist	39.8 ± 3.8	23.7 ± 2.8	13.2 ± 1.2	113.6 ± 41.9
OOD Embodiment(s)	Specialist	54.4 ± 8.2	27.0 ± 5.5	5.6 ± 3.4	144.8 ± 24.5
	Generalist	49.8 ± 8.3	29.4 ± 15.7	9.2 ± 6.1	102.9 ± 44.3

TABLE II: CrossSafe specialists vs. generalists.

collision rate (CR) of the unfiltered nominal planner from 64.1% to 39.8% on in-distribution embodiments (InD) and to 49.8% on the out-of-distribution embodiment (OOD). The Force metric also falls from 179.6 N to 113.6 N InD and 102.9 N OOD. Across the 25 task–embodiment pairs, CrossSafe lowers CR relative to the nominal policy in 21/25 InD and in 19/25 OOD pairs. Relative to the nominal controller, CrossSafe reduces the mean CR over the five tasks for every in-distribution embodiment. CR decreases on all five tasks for ARX-X5 (mean CR decreases from 75.2% to 42.5%) and Aloha-AgileX (66.4% to 13.3%), on 4 of 5 tasks for Piper (51.6% to 29.3%) and Franka-Panda (66.0% to 57.1%), and on 3 of 5 tasks for UR5-WSG (61.2% to 56.9%). For OOD evaluations, mean CR decreases on four of the five held-out embodiments. CR decreases on 5 of 5 tasks for ARX-X5 (mean CR decreases from 75.2% to 40.0%) and Aloha-AgileX (66.4% to 44.4%), and on 4 of 5 for Franka-Panda (66.0% to 60.4%) and UR5-WSG (61.2% to 48.4%), while on Piper mean CR increases from 51.6% to 56.0%, with CR reduced on only 1 of 5 tasks.

b) CrossSafe generalizes to embodiments unseen during safety-filter training: On the OOD embodiment, CrossSafe reduces CR by 14.3 percentage points and Force from 179.6 N to 102.9 N (Table I), while intervening on only 9.2% of control steps. Among the held-out folds, Franka-Panda provides the clearest test of generalization across embodiments. Franka-Panda has seven DoFs per arm while the other four embodiments have six, so in that fold every trainable component of the filter is trained exclusively on 6-DoF arms and the safe policy must then output an additional joint angle displacement per arm at test time. Despite these differences in robot morphology and action dimension, CrossSafe still lowers mean CR from 66.0% to 60.4% (it lowers it on 4 out of 5 tasks) while raising mean SR from 40.4% to 45.2%, and attains the lowest CR of the five methods on the held-out Franka-Panda embodiment. When the Franka-Panda embodiment is held out, CrossSafe’s three ablation variants (CrossSafe w/o Geo, Aug; CrossSafe w/o Geo; CrossSafe

w/o Aug) all raise mean CR above the nominal controller’s CR (from 66.0% to 76.0%, 79.2%, and 66.4%, respectively). These results demonstrate that the different components of CrossSafe helped it generalize from training embodiments with six DoFs per arm to an unseen embodiment with seven.

c) Geometry-aware attention and safety-feature augmentation enable generalization: Table I compares CrossSafe with CrossSafe w/o Geo, CrossSafe w/o Aug, and CrossSafe w/o Geo, Aug to assess geometry-aware attention and safety-feature augmentation. All four CrossSafe variants achieve lower mean CR and Force than the nominal controller in both InD and OOD evaluations. When safety-feature augmentation is omitted, CrossSafe w/o Aug achieves lower mean OOD CR than CrossSafe w/o Geo, Aug. When it is included, CrossSafe achieves lower mean OOD CR than CrossSafe w/o Geo. In both comparisons, the model using geometry-aware attention achieves lower mean OOD CR than its counterpart using standard cross-attention. With geometry-aware attention retained in both models, CrossSafe w/o Aug and CrossSafe achieve nearly identical mean OOD CRs (49.7% and 49.8%, respectively), while CrossSafe achieves a higher SR (29.4% vs. 23.3%), lower Force (102.9 N vs. 133.5 N), and the same mean IR. These results support our hypothesis that combining geometry-aware attention and safety-feature augmentation benefits generalization to unseen embodiments.

d) CrossSafe achieves lower collision rates and higher task success rates than X-VLA-Safe: Compared with X-VLA-Safe, CrossSafe achieves lower mean CR and Force and higher mean SR in both InD and OOD evaluations (Table I). Relative to nominal, CrossSafe cuts CR by 37.9% and 22.3% and contact force by 36.7% and 42.7% for InD and OOD, respectively, while giving up only 26.4% and 8.7% of task success. X-VLA-Safe reduces CR by only 21.4% InD and 16.1% OOD, and does so while halving task success (47.2% and 48.1% drops, respectively) and raising Force 5.6% and 37.5% above nominal.

e) Training across more embodiments improves generalization to OOD settings with minimal effects on InD performance: Compared to specialists, the generalists improve all OOD outcome metrics: CR 49.8% vs. 54.4%, SR 29.4% vs. 27.0%, and Force 102.9 N vs. 144.8 N. Jointly training the safety filter across embodiments therefore yields a filter that exhibits better generalization. For the InD task–embodiment pairs, against the specialists, the generalists attain lower CR in 13 of 25 and higher SR in 15 of 25. The generalists’

mean InD SR is also higher (23.7% vs. 18.7%), although mean InD CR is slightly higher (39.8% vs. 37.6%).

VII. CONCLUSION

We hypothesized that the reasoning required to satisfy a safety constraint is largely shared across robots, while the action that realizes it depends on each robot’s morphology, kinematics, and dynamics. CrossSafe instantiates this idea with a representation of the robot as a variable-length sequence of link tokens encoding kinematics and nearby scene geometry, allowing one HJ critic and safe policy to be learned across bimanual manipulators with different degrees of freedom. The representation itself is not tied to the HJ formulation and can be used to design other latent safety filters, including ones based on neural control barrier functions. In RoboTwin 2.0, CrossSafe reduces the nominal policy’s collision rate and contact force both on training embodiments and on embodiments unseen during the training of the safety filter. Two limitations remain. As a learned filter, CrossSafe provides no formal guarantee, and filtered collision rates remain relatively high. Future work includes calibrating the learned value function using conformal prediction or scenario optimization to obtain probabilistic guarantees, training across a larger and more diverse set of robot embodiments and tasks, and hardware validation.

REFERENCES

- [1] Open X-Embodiment Collaboration, “Open x-embodiment: Robotic learning datasets and rt-x models,” *arXiv preprint arXiv:2310.08864*, 2023.
- [2] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, C. Xu, J. Luo, T. Kreiman, Y. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine, “Octo: An open-source generalist robot policy,” in *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024.
- [3] R. Doshi, H. Walke, O. Mees, S. Dasari, and S. Levine, “Scaling cross-embodied learning: One policy for manipulation, navigation, locomotion and aviation,” *arXiv preprint arXiv:2408.11812*, 2024.
- [4] J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang, Y. Feng, Y. Zheng, J. Zou, Y. Chen, J. Zeng, Y.-Q. Zhang, J. Pang, J. Liu, T. Wang, and X. Zhan, “X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model,” *arXiv preprint arXiv:2510.10274*, 2025.
- [5] X. Lin, T. Lin, Y. Du, H. Xie, Y. Jin, J. Li, S. Wu, Q. Wang, M. Li, M. Zhao, Z. Li, C. Huang, H. Bi, L. Huang, and Z. Su, “HoloBrain-0 technical report,” *arXiv preprint arXiv:2602.12062*, 2026.
- [6] H. Li, G. Zhao, Y. Liu, H. Hou, G. Ye, T. Fang, C. Liu, S. Huang, J. Liu, X. Wang, and H. Li, “ACE-Ego-0: Unifying ego-centric human and robotic data for VLA pretraining,” *arXiv preprint arXiv:2606.17200*, 2026.
- [7] B. Han, Y.-W. Chao, E. Coumans, C. Eppner, B. Sundaralingam, J. Deng, S. Birchfield, and A. Murali, “GraspGen-X: Cross-embodiment 6-dof diffusion-based grasping,” *arXiv preprint arXiv:2606.00998*, 2026.
- [8] Y. Wu, Y. Lin, W. Lao, Y. Lin, Y.-L. Wei, W.-S. Zheng, and A. Wu, “DexGrasp-Zero: A morphology-aligned policy for zero-shot cross-embodiment dexterous grasping,” *arXiv preprint arXiv:2603.16806*, 2026.
- [9] M. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “Openvla: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [10] S. Wang, K. Lv, M. Yu, and X. Li, “EmbodiSteer: Steering embodiment-agnostic visuomotor policies with joint-space guidance for zero-shot cross-embodiment deployment,” *arXiv preprint arXiv:2606.12965*, 2026.
- [11] A. Beaudin, H. Krasowski, K. Nagpal, S. A. Seshia, M. Arcak, and N. Mehr, “Any-body guard: Universal safeguarding for manipulation policies via action masking,” *arXiv preprint arXiv:2606.22278*, 2026.
- [12] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu, *et al.*, “Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation,” *arXiv preprint arXiv:2506.18088*, 2025.
- [13] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, *et al.*, “ $\pi_{0.5}$: a vision-language-action model with open-world generalization,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.16054>
- [14] Y. Zhang, S. Zhang, Y. Shen, S. Dong, J. Deng, X. Zhang, Y. Gao, J. Wu, X. Nie, Z. Cheng, J. Ji, Y. Zhang, X. Zhang, and J. Pan, “GEAR-VLA: Learning geometry-aware action representations for generalizable robotic manipulation,” *arXiv preprint arXiv:2606.08530*, 2026.
- [15] T. Wu, S. Li, J. Gong, C. Guo, X. Li, S. Mu, and W. Ding, “CEI: A unified interface for cross-embodiment visuomotor policy learning in 3d space,” *arXiv preprint arXiv:2601.09163*, 2026.
- [16] A. Sridhar, J. Gao, J. Yang, J. Mercat, S. Belkhal, and D. Sadigh, “Cross-embodiment transfer via behavior-aligned representations,” *arXiv preprint arXiv:2607.27549*, 2026.
- [17] H. Li, G. Li, Y. Feng, C. Zhao, Z. Wang, Y. Li, Q. Wei, S. Bao, H. Shen, Y. Zhao, T. Yang, and J. Zhang, “Training vision-language-action models with dense embodied chain-of-thought supervision,” *arXiv preprint arXiv:2606.30552*, 2026.
- [18] A. Patel and S. Song, “GET-Zero: Graph embodiment transformer for zero-shot embodiment generalization,” *arXiv preprint arXiv:2407.15002*, 2024.
- [19] W. Niu, Q. Ke, Y. Sun, H. Sun, J. Xu, M. Ma, R. Hu, and F. Sun, “EAGG: Embodiment-aligned grasp generation via geometry-aware graph conditioning,” *arXiv preprint arXiv:2606.18092*, 2026.
- [20] N. Bohlinger, G. Czechmanowski, M. Krupka, P. Kicki, K. Walas, J. Peters, and D. Tateo, “One policy to run them all: an end-to-end learning approach to multi-embodiment locomotion,” *arXiv preprint arXiv:2409.06366*, 2024.
- [21] M. H. Danesh, C. Li, A. Abyaneh, A. Houssaini, K. Ellis, G. Berseth, M. Hutter, and H.-C. Lin, “Morphology-conditioned world model for cross-embodiment quadrupedal locomotion,” *arXiv preprint arXiv:2604.08780*, 2026.
- [22] L. Zha, A. J. Hancock, M. Zhang, T. Yin, Y. Huang, D. Shah, A. Z. Ren, and A. Majumdar, “LAP: Language-action pre-training enables zero-shot cross-embodiment transfer,” *arXiv preprint arXiv:2602.10556*, 2026.
- [23] M. Pisen, G. Tevet, and C. K. Liu, “Cloak: Zero-shot cross-embodiment manipulation by masking the end-effector from the VLA,” *arXiv preprint arXiv:2606.22836*, 2026.
- [24] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [25] S. Bansal, M. Chen, S. Herbert, and C. J. Tomlin, “Hamilton-jacobi reachability: A brief overview and recent advances,” in *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*. IEEE, 2017, pp. 2242–2253.
- [26] O. So, Z. Serlin, M. Mann, J. Gonzales, K. Rutledge, N. Roy, and C. Fan, “How to train your neural control barrier function: Learning safety filters for complex input-constrained systems,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 11 532–11 539.
- [27] I. Tabbara and H. Sibai, “Learning conservative neural control barrier functions from offline data,” *arXiv preprint arXiv:2505.00908*, 2025.
- [28] J. F. Fisac, N. F. Lugovoy, V. Rubies-Royo, S. Ghosh, and C. J. Tomlin, “Bridging hamilton-jacobi safety analysis and reinforcement learning,” in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8550–8556.
- [29] K. Nakamura, L. Peters, and A. Bajcsy, “Generalizing safety beyond collision-avoidance via latent-space reachability analysis,” in *Proceedings of Robotics: Science and Systems*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.00935>
- [30] I. Tabbara, Y. Yang, A. Hamzeh, M. Astafyev, and H. Sibai, “Designing latent safety filters using pre-trained vision models,” *arXiv preprint arXiv:2509.14758*, 2025.
- [31] Y. Li and M. Chen, “Hjrho: Hamilton-jacobi reachability with neural operators,” *arXiv preprint arXiv:2504.19989*, 2025.

- [32] K. Long, K. M. B. Lee, N. Raicevic, N. Attasseri, M. Leok, and N. Atanasov, "Neural configuration-space barriers for manipulation planning and control," *arXiv preprint arXiv:2503.04929*, 2025.
- [33] M. Tayal, M. Tayal, and R. Prakash, "Safe flow q-learning: Offline safe reinforcement learning with reachability-based flow policies," *arXiv preprint arXiv:2603.15136*, 2026.
- [34] O. So, W. Sharpless, S. Herbert, and C. Fan, "Value functions for temporal logic: Optimal policies and safety filters," *arXiv preprint arXiv:2605.01051*, 2026.
- [35] I. Tabbara, Y. Yang, and H. Sibai, "Towards general language-conditioned latent safety filters," *arXiv preprint arXiv:2608.00315*, 2026.
- [36] A. Lin and S. Bansal, "Verification of neural reachable tubes via scenario optimization and conformal prediction," in *Learning for Dynamics and Control Conference (LADC)*, ser. Proceedings of Machine Learning Research, vol. 242. PMLR, 2024, pp. 719–731.
- [37] I. Tabbara, Y. Yang, and H. Sibai, "Statistically assuring safety of control systems using ensembles of safety filters and conformal prediction," *arXiv preprint arXiv:2511.07899*, 2025.
- [38] J. Seo, K. Nakamura, and A. Bajcsy, "Uncertainty-aware latent safety filters for avoiding out-of-distribution failures," in *Conference on Robot Learning (CoRL)*, 2025.
- [39] M. Tayal, A. Singh, P. Jagtap, and S. Kolathaya, "Cp-ncbf: A conformal prediction-based approach to synthesize verified neural control barrier functions," *arXiv preprint arXiv:2503.17395*, 2025.
- [40] S. Huriot, I. Tabbara, and H. Sibai, "Safe control using learned safety filters and adaptive conformal inference," in *Proceedings of The 8th Annual Learning for Dynamics and Control Conference*, ser. Proceedings of Machine Learning Research, G. Sukhatme, L. Lindemann, S. Tu, A. Wierman, and N. Atanasov, Eds., vol. 331. PMLR, 17–19 Jun 2026, pp. 833–847. [Online]. Available: <https://proceedings.mlr.press/v331/huriot26a.html>
- [41] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International conference on machine learning*. Pmlr, 2018, pp. 1861–1870.
- [42] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. PMLR, 10–15 Jul 2018, pp. 1587–1596. [Online]. Available: <https://proceedings.mlr.press/v80/fujimoto18a.html>
- [43] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, *et al.*, "Soft actor-critic algorithms and applications," *arXiv preprint arXiv:1812.05905*, 2018.

1 Simulation environment	10
1.1 Control frequency	10
1.2 Embodiments	10
1.3 Tasks	10
1.4 Obstacle placement	10
1.5 Failure function	10
2 Safety critic and policy: implementation details	10
2.1 Soft actor-critic training	10
2.2 Safe actor head	10
2.3 Data collection	10
2.4 Parameter count	10
2.5 CrossSafe training algorithm	10
2.6 Hyperparameters	10
3 Evaluation	11
3.1 Scene determinism	11
3.2 Extended results	11

A. Simulation environment

1) *Control frequency*: All experiments run in RoboTwin 2.0 on top of SAPIEN. The physics timestep is fixed at 250 Hz and the control loop at 25 Hz, so one control tick is exactly ten physics substeps and the control period is $\Delta t = 0.04$ s. Episodes are capped at 450 control ticks (18 s of simulated time).

2) *Embodiments*: All embodiments are dual-arm configurations of the corresponding RoboTwin 2.0 robot, with the two arms treated as one system. Franka-Panda has seven actuated joints per arm; the remaining four have six.

Joint action limits are derived from the URDF of each robot: $a_i^{\max} = \dot{q}_i^{\lim} \Delta t$.

3) *Tasks*: We use five bimanual RoboTwin 2.0 manipulation tasks: *Place Bread in Basket*, *Place Container on Plate*, *Stack Two Blocks*, *Place Burger & Fries*, and *Stack Two Bowls*. Task objectives, success criteria, and scene randomization are inherited unchanged from RoboTwin 2.0. Background randomization is left at the benchmark defaults and turned off. Our only modification is the addition of one static obstacle per scene, described next.

4) *Obstacle placement*: The obstacle is the RoboTwin-OD 068_boxdrink mesh, a box of roughly $11.0 \times 15.4 \times 11.6$ cm, spawned as a static actor resting on the table. Let p_{pick} and p_{place} be the table-plane positions of the picked object and the place target. The obstacle is centered at

$$p(\rho) = (1 - \rho) p_{\text{pick}} + \rho p_{\text{place}}, \quad \rho \sim \mathcal{U}[0.22, 0.48], \quad (12)$$

drawn once per episode, so that $\rho = 0$ is the pick pose and $\rho = 1$ the place pose.

In *on-path* mode the obstacle sits at $p(\rho)$; in *off-path* mode it is offset perpendicular to the segment. During collection the mode is drawn independently of the task and embodiment, *on-path* with probability 0.7, so that filter engagement is not confounded with the obstacle being in the way. All evaluation scenes are *on-path*.

5) *Failure function*: The failure function is the signed distance between the robot system and the obstacle,

$$h(s) = \min \text{dist}(\mathcal{B}(s) \cup \mathcal{P}(s), \mathcal{O}), \quad (13)$$

where $\mathcal{B}(s)$ is the set of collision spheres on the moving links of both arms, $\mathcal{P}(s)$ is the grasped object if one is held, and $\mathcal{O}$ is the oriented bounding box of the obstacle.

B. Safety critic and policy: implementation details

1) *Soft actor-critic training*: We instantiate two critic heads (twin critics) [42] with separate transformer blocks and separate value heads, both using the same encoded latent state. We train the critics and the safe policy with SAC [43] adapted to the HJ reachability setting: the critics regress the discounted avoid target y_t of Eq. (1). The target carries no entropy bonus, since the soft value $\alpha \log \sum_a \exp(Q/\alpha) \geq \max_a Q$ and larger Q means safer, so a soft target would make the value function optimistic about safety. The entropy term is therefore kept only in the policy loss, where it acts as an exploration regularizer with α annealed from 0.2 to 0.02 over the first 20,000 gradient steps. Target networks are Polyak averaged with $\tau = 0.005$.

2) *Safe actor head*: The actor is a single linear map $\text{Linear}(256 \rightarrow 2)$ applied to every actuated link token, producing a mean and a log standard deviation. The log standard deviation is clamped to $[-5, 0]$. Actions are sampled with the reparameterization trick, squashed by $\tanh$, scattered into the action vector by joint index, and only then scaled by the per-joint limit $a_i^{\max}$, so $|a_i| \leq a_i^{\max}$ holds by construction and every predicted action is feasible.

3) *Data collection*: First, a single shared warmup buffer is collected once with the nominal controller only, using a fixed cyclic schedule over the shuffled (task, embodiment) product so that every one of the 25 combinations appears exactly 20 times, for 500 trajectories. Exact balance matters here: i.i.d. sampling leaves the combinations badly unbalanced at this scale. Only episodes in which the task succeeds are retained. The same warmup buffer is reused by every split and every ablation; the trainer restricts sampling by embodiment identifier rather than re-collecting per split.

Training then alternates between 1024 gradient steps and the collection of three fresh episodes with the current weights. Within a collection round, an episode runs with the safety filter active with probability 0.8 and nominal-only otherwise.

4) *Parameter count*: Table III reports trainable parameter counts. The frozen HoloBrain-0 encoders are excluded; they receive no gradient.

5) *CrossSafe training algorithm*: Algorithm (1) shows how the safe actor and critic are trained.

6) *Hyperparameters*: Table IV lists every hyperparameter. All methods, ablations, folds, and specialists use identical values; the only differences across runs are the embodiment pool used for training the safety filter components.

TABLE III: Trainable parameters (frozen HoloBrain-0 excluded).

Module	Parameters
Fusion of per-link tokens	202,752
Geometry-aware transformer (3 blocks)	3,177,720
Safe actor head	514
Twin safety critic (2× [2 blocks + value head])	4,373,666
CrossSafe (total)	7,754,652
CrossSafe w/o Geo	7,708,676
CrossSafe w/o Aug	7,750,556
CrossSafe w/o Geo, Aug	7,704,580

Algorithm 1 CrossSafe training

Require: embodiment pool $\mathcal{E}$, shared warmup buffer $\mathcal{D}_0$

- 1: $\mathcal{D} \leftarrow$ transitions of $\mathcal{D}_0$ whose embodiment is in $\mathcal{E}$
- 2: initialize encoder ψ , critic θ , actor ϕ
- 3: target copies $\psi^- \leftarrow \psi$, $\theta^- \leftarrow \theta$
- 4: **for** round $r = 1$ **to** R **do**
- 5: **for** 1024 gradient steps **do**
- 6: sample a batch of 64 transitions from $\mathcal{D}$
- 7: encode $z \leftarrow E_\psi(s)$ and $z' \leftarrow E_{\psi^-}(s')$
- 8: update θ and ψ using Eq. (1)
- 9: update ϕ to maximize $Q_\theta(z, \pi_\phi(z))$, holding θ and ψ fixed
- 10: $\psi^- \leftarrow (1 - \tau)\psi^- + \tau\psi$; $\theta^- \leftarrow (1 - \tau)\theta^- + \tau\theta$
- 11: **end for**
- 12: save checkpoint
- 13: **for** 3 episodes, cycling over tasks and embodiments in $\mathcal{E}$ **do**
- 14: with probability 0.8 roll out with the safety filter, else roll out the nominal controller with random action perturbations
- 15: append the episode’s transitions to $\mathcal{D}$
- 16: **end for**
- 17: **end for**

C. Evaluation

1) *Scene determinism:* All methods, all ablations, and the unfiltered nominal controller are evaluated on identical scenes, and re-running an evaluation reproduces the same results. Each (task, embodiment) cell logs exactly 50 episodes: planner failures that produce an empty trace, and obstacle-spawn failures, redraw a fresh deterministic scene from a derived seed until a real episode completes, with a cap on attempts. The safe policy is evaluated deterministically, using the pre-activation mean with no sampling.

2) *Extended results:* Tables V and VI report collision and success rate for every task–embodiment pair.

TABLE IV: Hyperparameters. Values in the upper block change what V means and are frozen across all runs.

<i>Problem definition</i>	
Physics frequency	250 Hz
Control frequency	25 Hz
Control period Δt	0.04 s
Action-limit fraction κ	1.0
Soft-min temperature	0.02 m ($T = 0.4$ scaled)
Discount γ	0.9 (fixed)
Max episode length	450 ticks (18 s)
Obstacle model	068_boxdrink
<i>Off-path</i> fraction (data collection)	0.3
<i>Architecture</i>	
link token width	256
Attention heads	8
Feed-forward width	1024
Normalization	RMSNorm (pre-norm)
Activation	SiLU
Scene tokens N_{scene}	1200 (3 cams $\times$ 400)
Image resolution	320 $\times$ 256
<i>Optimization</i>	
Optimizer	Adam
Learning rate	3×10^{-4}
Batch size	64
Target Polyak τ	0.005
Entropy coefficient α	0.2 $\rightarrow$ 0.02 over 20k steps
Pre-activation penalty λ	10^{-3}
log σ clamp	$[-5, 0]$
Gradient-norm clip	10
Gradient steps per round	1024
Episodes collected per round	3
Filter-episode fraction	0.8
Perturbation probability	0.05
Warmup trajectories	500 (20 per pair)

TABLE V: In-distribution collision and success rate per task and embodiment, shown as CR/SR (both %). *Nominal* is the unfiltered cuRobo planner and is training-independent. *X-VLA-Safe* and the four *CrossSafe* variants are the leave-one-embodiment-out models: each cell averages the four models that had that embodiment in their training pool ($4 \times 50 = 200$ episodes). *Specialist* is a CrossSafe model trained on that embodiment alone and evaluated on it (50 episodes). Among the learned methods, bold marks the lowest CR and the highest SR in each row. *Task avg.* averages the five embodiments; *Overall average* averages all 25 pairs.

Task	Emb.	<i>Nominal</i>	X-VLA	Specialist	CrossSafe w/o Geo, Aug	CrossSafe w/o Geo	CrossSafe w/o Aug	CrossSafe
Place Bread in Basket	Piper	74.0/28.0	46.0/8.5	38.0/16.0	47.5/15.0	53.5/ 17.0	48.5/16.5	37.5 /15.0
	Franka-Panda	98.0/10.0	85.5/4.0	66.0/0.0	62.5/ 7.0	74.0/4.5	60.0 /1.5	80.0/0.5
	ARX-X5	88.0/24.0	66.0/5.5	30.0 /6.0	40.0/ 34.5	63.0/26.0	46.5/25.0	63.5/14.5
	UR5-WSG	98.0/10.0	87.0/12.0	96.0/10.0	77.0 / 46.5	82.0/34.0	83.5/27.5	93.0/16.5
	Aloha-AgileX	76.0/16.0	73.0/6.0	0.0 /0.0	43.0/ 12.0	51.0/7.0	44.5/5.0	13.0/0.0
	<i>Task avg.</i>	86.8/17.6	71.5/7.2	46.0 /6.4	54.0/ 23.0	64.7/17.7	56.6/15.1	57.4/9.3
Place Container on Plate	Piper	38.0/0.0	12.0/1.5	10.0 / 18.0	15.5/11.5	32.5/6.0	10.0 /13.5	25.5/7.0
	Franka-Panda	86.0/8.0	51.5/ 28.0	48.0/20.0	53.5/25.0	62.5/21.5	46.5 /19.5	69.0/25.0
	ARX-X5	76.0/2.0	45.0/9.5	28.0/18.0	18.5 / 37.0	42.5/20.5	33.5/21.0	42.0/14.0
	UR5-WSG	70.0/4.0	47.0/3.0	50.0/16.0	44.5/47.5	37.0/ 48.5	35.0 /26.5	48.5/25.5
	Aloha-AgileX	76.0/6.0	42.0/12.0	28.0/10.0	17.0 / 36.5	33.0/13.0	23.5/16.5	26.5/11.0
	<i>Task avg.</i>	69.2/4.0	39.5/10.8	32.8/16.4	29.8/ 31.5	41.5/21.9	29.7 /19.4	42.3/16.5
Stack Two Blocks	Piper	54.0/60.0	57.0/32.5	20.0 / 62.0	30.5/59.0	38.5/43.5	20.0 /57.5	41.0/53.5
	Franka-Panda	32.0/90.0	45.5/60.0	42.0/46.0	53.0/61.0	42.5/53.0	26.5 / 64.0	40.5/58.5
	ARX-X5	82.0/72.0	66.5/40.0	60.0/38.0	48.0/ 65.5	55.5/63.0	56.5/50.5	47.0 /61.0
	UR5-WSG	24.0/84.0	31.5/72.5	50.0/62.0	35.5/81.0	35.5/78.5	38.5/ 83.5	31.0 /80.5
	Aloha-AgileX	80.0/44.0	64.0/1.5	28.0/0.0	29.5/ 2.0	40.5/0.0	29.5/1.0	11.5 /0.0
	<i>Task avg.</i>	54.4/70.0	52.9/41.3	40.0/41.6	39.3/ 53.7	42.5/47.6	34.2 /51.3	34.2 /50.7
Place Burger & Fries	Piper	84.0/18.0	38.0/14.5	24.0 /40.0	30.5/36.5	51.5/27.5	27.0/ 46.0	31.0/31.0
	Franka-Panda	100.0/6.0	79.0/0.5	66.0 /0.0	83.5/0.5	74.0/1.0	70.0/0.0	83.0/ 1.5
	ARX-X5	100.0/0.0	61.0/1.5	58.0/0.0	32.5 / 16.5	47.5/8.5	42.0/5.5	51.0/2.5
	UR5-WSG	100.0/2.0	88.0/8.5	100.0/4.0	88.5/ 30.0	93.5/17.0	86.0 /25.0	95.0/7.0
	Aloha-AgileX	70.0/6.0	64.0/0.0	20.0/0.0	25.5/ 7.0	31.0/0.0	39.0/3.0	15.5 /0.0
	<i>Task avg.</i>	90.8/6.4	66.0/5.0	53.6/8.8	52.1 / 18.1	59.5/10.8	52.8/15.9	55.1/8.4
Stack Two Bowls	Piper	8.0/28.0	12.5/2.0	10.0/ 10.0	7.0/9.0	14.5/4.0	5.0 /8.0	11.5/6.5
	Franka-Panda	14.0/88.0	7.5 / 55.0	12.0/42.0	24.5/40.5	20.0/35.0	17.5/51.0	13.0/54.5
	ARX-X5	30.0/66.0	44.0/23.0	4.0 /26.0	16.5/46.5	20.0/31.0	16.0/36.5	9.0/ 47.5
	UR5-WSG	14.0/86.0	21.5/22.5	48.0/24.0	23.5/56.0	27.5/35.5	17.0 /44.5	17.0 / 58.5
	Aloha-AgileX	30.0/48.0	24.0/ 1.0	4.0/0.0	10.0/0.0	11.5/0.5	4.5/0.5	0.0 /0.0
	<i>Task avg.</i>	19.2/63.2	21.9/20.7	15.6/20.4	16.3/30.4	18.7/21.2	12.0/28.1	10.1 / 33.4
Overall average		64.1/32.2	50.4/17.0	37.6/18.7	38.3/ 31.3	45.4/23.8	37.1 /26.0	39.8/23.7

TABLE VI: Zero-shot collision and success rate on held-out embodiments, shown as CR/SR (both %). Conditions are as in Table V, but each *X-VLA-Safe* and *CrossSafe* cell comes from the single model for which that embodiment was excluded from safety-filter training (50 episodes), and each *Specialist* cell averages the four single-embodiment models that did not train on it ($4 \times 50 = 200$ episodes). Among the learned methods, bold marks the lowest CR and the highest SR in each row.

Task	Emb.	<i>Nominal</i>	X-VLA	Specialist	CrossSafe w/o Geo, Aug	CrossSafe w/o Geo	CrossSafe w/o Aug	CrossSafe
Place Bread in Basket	Piper	74.0/28.0	24.0/6.0	37.0/9.5	68.0/12.0	76.0/ 26.0	64.0/16.0	74.0/18.0
	Franka-Panda	98.0/10.0	94.0/18.0	95.5/11.5	98.0/ 22.0	98.0/6.0	98.0/10.0	96.0/6.0
	ARX-X5	88.0/24.0	64.0/4.0	66.5/25.0	32.0/40.0	36.0/28.0	38.0/18.0	42.0/ 44.0
	UR5-WSG	98.0/10.0	82.0/8.0	82.0/9.0	92.0/4.0	66.0/22.0	72.0/4.0	82.0/0.0
	Aloha-AgileX	76.0/16.0	82.0/8.0	24.5/0.0	60.0/8.0	54.0/ 22.0	60.0/4.0	66.0/4.0
	<i>Task avg.</i>	86.8/17.6	69.2/8.8	61.1/11.0	70.0/17.2	66.0/ 20.8	66.4/10.4	72.0/14.4
Place Container on Plate	Piper	38.0/0.0	18.0/0.0	47.0/2.5	34.0/2.0	42.0/2.0	48.0/2.0	52.0/ 4.0
	Franka-Panda	86.0/8.0	76.0/20.0	91.5/21.0	84.0/50.0	98.0/20.0	78.0/20.0	70.0/58.0
	ARX-X5	76.0/2.0	54.0/8.0	58.5/20.5	30.0/ 44.0	38.0/38.0	34.0/12.0	22.0/30.0
	UR5-WSG	70.0/4.0	44.0/8.0	82.5/20.0	80.0/22.0	80.0/24.0	82.0/10.0	78.0/ 34.0
	Aloha-AgileX	76.0/6.0	54.0/16.0	62.5/14.0	40.0/ 44.0	28.0/22.0	18.0/10.0	30.0/12.0
	<i>Task avg.</i>	69.2/4.0	49.2/10.4	68.4/15.6	53.6/ 32.4	57.2/21.2	52.0/10.8	50.4/27.6
Stack Two Blocks	Piper	54.0/60.0	42.0/20.0	55.0/61.0	52.0/60.0	54.0/ 66.0	52.0/64.0	60.0/ 66.0
	Franka-Panda	32.0/90.0	34.0/78.0	36.0/ 86.0	76.0/80.0	40.0/74.0	36.0/82.0	34.0/84.0
	ARX-X5	82.0/72.0	52.0/50.0	61.0/68.0	30.0/ 80.0	26.0/58.0	52.0/40.0	80.0/28.0
	UR5-WSG	24.0/84.0	44.0/50.0	33.0/69.0	42.0/72.0	34.0/ 74.0	24.0/56.0	20.0/72.0
	Aloha-AgileX	80.0/44.0	82.0/0.0	33.5/0.0	60.0/ 4.0	78.0/2.0	68.0/0.0	64.0/0.0
	<i>Task avg.</i>	54.4/70.0	50.8/39.6	43.7/56.8	52.0/ 59.2	46.4/54.8	46.4/48.4	51.6/50.0
Place Burger & Fries	Piper	84.0/18.0	26.0/0.0	78.5/23.5	80.0/16.0	84.0/ 30.0	84.0/24.0	82.0/28.0
	Franka-Panda	100.0/6.0	100.0/4.0	100.0/1.5	100.0/ 10.0	100.0/4.0	100.0/4.0	96.0/2.0
	ARX-X5	100.0/0.0	74.0/2.0	84.0/9.0	30.0/ 44.0	44.0/10.0	28.0/8.0	42.0/12.0
	UR5-WSG	100.0/2.0	90.0/2.0	92.5/1.5	84.0/12.0	84.0/ 18.0	84.0/0.0	56.0/0.0
	Aloha-AgileX	70.0/6.0	84.0/ 18.0	43.5/1.0	48.0/6.0	54.0/14.0	56.0/0.0	58.0/0.0
	<i>Task avg.</i>	90.8/6.4	74.8/5.2	79.7/7.3	68.4/ 17.6	73.2/15.2	70.4/7.2	66.8/8.4
Stack Two Bowls	Piper	8.0/28.0	6.0/2.0	11.5/27.5	8.0/ 36.0	8.0/26.0	10.0/30.0	12.0/30.0
	Franka-Panda	14.0/88.0	10.0/62.0	16.0/79.5	22.0/70.0	60.0/40.0	20.0/ 80.0	6.0/76.0
	ARX-X5	30.0/66.0	50.0/10.0	27.5/43.5	12.0/ 50.0	8.0/48.0	10.0/40.0	14.0/36.0
	UR5-WSG	14.0/86.0	34.0/24.0	29.5/69.5	12.0/82.0	24.0/76.0	22.0/48.0	6.0/88.0
	Aloha-AgileX	30.0/48.0	26.0/0.0	10.0/0.5	14.0/ 4.0	20.0/0.0	4.0/0.0	4.0/4.0
	<i>Task avg.</i>	19.2/63.2	25.2/19.6	18.9/44.1	13.6/ 48.4	24.0/38.0	13.2/39.6	8.4/46.8
Overall average		64.1/32.2	53.8/16.7	54.4/27.0	51.5/ 35.0	53.4/30.0	49.7/23.3	49.8/29.4